Panduan Teknis 03 - Jupyter Notebook dan Google Colab

Dokumen ini menjelaskan cara menjalankan eksperimen Python melalui Jupyter Notebook dan Google Colab. Bagian ini dibuat untuk pembaca yang belum terbiasa dengan cell, kernel/runtime, path file, Google Drive, atau GPU cloud.

> **Urutan yang akan dilakukan:** (1) memilih notebook lokal atau Google Colab; (2) menjalankan notebook; (3) memahami urutan cell; (4) mengaktifkan GPU; (5) memasang dependency; (6) mount Google Drive; (7) menentukan folder project; (8) menjalankan eksperimen; (9) menyimpan hasil; dan (10) melakukan restart lalu menjalankan ulang bila diperlukan.

Langkah 1 - Pahami Kapan Menggunakan Notebook

File Python biasa (`.py`) menjalankan script dari awal sampai akhir. Notebook (`.ipynb`) membagi eksperimen menjadi beberapa **cell** yang dapat dijalankan satu per satu. Notebook cocok untuk materi hands-on karena penjelasan, kode, output, dan visualisasi dapat ditempatkan berdekatan.

Pilihan	Cocok untuk	Resource
Jupyter lokal	Eksperimen pada komputer sendiri, akses file/perangkat lokal	CPU/GPU komputer sendiri
Google Colab	Demo, pembelajaran, inference, eksperimen tanpa setup lokal berat	Runtime cloud yang bersifat sementara

Langkah 2 - Jalankan Jupyter Notebook Secara Lokal

Aktifkan virtual environment project terlebih dahulu, lalu pasang Jupyter:

```
python -m pip install notebook ipykernel
jupyter notebook
```

Browser akan membuka halaman Jupyter. Dari sana, buka file `.ipynb` atau buat notebook baru.

> **Jika perintah `jupyter` tidak ditemukan,** pastikan environment aktif dan package `notebook` memang terpasang di environment tersebut.

Langkah 3 - Kenali Code Cell dan Markdown Cell

Code cell dieksekusi oleh Python:

```
robot_name = "Franka"
print(robot_name)
```

Markdown cell berisi judul, penjelasan, catatan, atau formula. Markdown tidak dijalankan sebagai Python.

Untuk menjalankan code cell, gunakan tombol Run atau shortcut `Shift + Enter`.

Langkah 4 - Pahami Kernel/Runtime sebagai Memori

Variabel yang dibuat pada satu cell disimpan di memori dan dapat dipakai cell berikutnya.

```
# Cell 1
robot_name = "Franka"
```

```
# Cell 2
print(robot_name)
```

Jika Cell 2 dijalankan sebelum Cell 1 pada runtime baru, `robot_name` belum ada dan Python akan menghasilkan `NameError`.

> **Urutan cell penting.** Notebook bisa tampak berhasil karena variabel lama masih tersimpan. Untuk memastikan notebook benar-benar reproducible, restart kernel/runtime lalu jalankan lagi dari atas.

Langkah 5 - Susun Folder Project dengan Jelas

```
project/
├── demo.ipynb
├── requirements.txt
├── scene.jpg
└── results/
```

Struktur sederhana membuat notebook tahu dari mana input dibaca dan ke mana hasil disimpan.

Langkah 6 - Buka dan Hubungkan Google Colab

1. Buka Google Colab melalui browser.
2. Buat notebook baru atau buka file `.ipynb` dari Google Drive.
3. Tunggu sampai runtime terhubung.
4. Jalankan satu cell sederhana untuk memastikan Python aktif.

```
print("Colab is ready.")
```

Perlu diingat bahwa membuka notebook dari Google Drive tidak berarti runtime otomatis bekerja dari folder tempat notebook disimpan.

Langkah 7 - Aktifkan GPU Jika Eksperimen Membutuhkannya

Dari menu runtime Colab, pilih accelerator GPU jika tersedia. Setelah runtime terhubung ulang, verifikasi dengan PyTorch:

```
import torch

print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

> **Hasil yang diharapkan.** Untuk runtime GPU, `torch.cuda.is_available()` bernilai `True` dan nama GPU tampil.

Langkah 8 - Letakkan Cell Instalasi di Bagian Awal

Runtime Colab sudah berisi banyak package, tetapi versinya belum tentu cocok dengan repository. Pasang dependency yang memang dibutuhkan project.

```
!pip install -q transformers accelerate
```

Jika repository memiliki `requirements.txt`:

```
!pip install -q -r requirements.txt
```

Karena runtime dapat di-reset, cell instalasi perlu dijalankan kembali pada sesi baru. Jika instalasi mengubah package inti secara besar, restart runtime setelah instalasi bila dokumentasi repository menyarankannya.

Langkah 9 - Mount Google Drive untuk File yang Persisten

Storage `/content` bersifat sementara. Agar file project dan hasil tidak hilang saat runtime berakhir, mount Google Drive:

```
from google.colab import drive

drive.mount("/content/drive")
```

Setelah proses otorisasi selesai, My Drive biasanya tersedia di:

```
/content/drive/MyDrive/
```

Langkah 10 - Tentukan Folder Project Secara Eksplisit

Ini langkah yang sangat penting. Walaupun notebook dibuka dari Drive, current working directory Colab biasanya tetap `/content`. Tentukan path project dengan jelas:

```
from pathlib import Path
import os

PROJECT_DIR = Path(
    "/content/drive/MyDrive/EmbodiedAI/Handson"
)

if not PROJECT_DIR.exists():
    raise FileNotFoundError(PROJECT_DIR)

os.chdir(PROJECT_DIR)

print("Current directory:", Path.cwd())
```

> **Gunakan direct path jika sudah diketahui.** Tidak perlu melakukan recursive scan ke seluruh Google Drive hanya untuk mencari folder project. Direct path lebih cepat dan mudah direproduksi.

Langkah 11 - Verifikasi File Input Sebelum Inference

Sebelum menjalankan model, periksa bahwa file yang dibutuhkan memang ditemukan.

```
image_path = PROJECT_DIR / "scene.jpg"

if not image_path.exists():
    raise FileNotFoundError(image_path)

print("Image found:", image_path)
```

Langkah sederhana ini menghindari error yang sebenarnya hanya berasal dari path file yang salah.

Langkah 12 - Upload File Manual untuk Eksperimen Cepat

Jika hanya membutuhkan satu file sementara, upload manual lebih cepat daripada mengatur folder Drive:

```
from google.colab import files

uploaded = files.upload()
```

File yang di-upload berada pada runtime sementara. Untuk project yang terdiri dari banyak file, Google Drive lebih terstruktur.

Langkah 13 - Buat Folder results dan Simpan Output

```
from pathlib import Path

RESULTS_DIR = PROJECT_DIR / "results"
RESULTS_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

output_path = RESULTS_DIR / "prediction.txt"
output_path.write_text(
    "Inference completed.
",
    encoding="utf-8",
)

print("Result saved to:", output_path)
```

Karena `RESULTS_DIR` berada di Google Drive, hasil tetap tersimpan ketika runtime berakhir.

Langkah 14 - Jalankan Notebook dari Atas ke Bawah

Setelah semua cell selesai disusun, lakukan pengujian bersih:

1. restart runtime/session,

2. jalankan cell instalasi dan import,
3. mount Google Drive,
4. atur `PROJECT_DIR`,
5. muat input dan model,
6. jalankan inference/training,
7. simpan hasil.

Jika alur ini berjalan tanpa mengandalkan variabel lama, notebook lebih mudah dipakai pembaca lain.

Langkah 15 - Tangani Out of Memory Secara Bertahap

Model besar dapat melebihi VRAM. Jika muncul `CUDA out of memory`, ubah satu hal dalam satu waktu:

1. kurangi batch size,
2. kurangi resolusi image,
3. gunakan `torch.inference_mode()` untuk inference,
4. gunakan FP16 jika model mendukung,
5. gunakan quantization 8-bit atau 4-bit hanya jika kompatibel dengan model dan stack library.

Jika object besar sudah tidak digunakan:

```
import gc
import torch

# Setelah object dihapus menggunakan del ...
gc.collect()
torch.cuda.empty_cache()
```

Langkah 16 - Restart Runtime Saat State Lama Mengganggu

Mengubah kode konfigurasi tidak otomatis mengubah object model yang sudah dimuat. Jika model sebelumnya dibuat dengan dtype, quantization, atau dependency lama, restart runtime lalu jalankan lagi dari awal agar state benar-benar bersih.

Langkah 17 - Bedakan Google Colab dan Google Cloud Platform

Google Colab adalah layanan notebook siap pakai yang cocok untuk eksperimen. Google Cloud Platform (GCP) adalah platform cloud yang lebih luas, misalnya untuk membuat virtual machine, storage, container, atau GPU instance yang dikelola sendiri. Untuk hands-on dasar pada buku ini, Colab biasanya sudah cukup dan GCP tidak perlu disiapkan.

Langkah 18 - Gunakan Pola Notebook yang Konsisten

Urutan notebook yang mudah diikuti biasanya adalah: **(1) install dependency; (2) import; (3) mount Drive; (4) set path; (5) load input; (6) load model; (7) run; dan (8) save result**. Dengan urutan ini, pembaca yang baru mengenal notebook dapat mengikuti proses dari atas ke bawah tanpa menebak state yang dibutuhkan.